Penetration Test Report

Target: TryHackMe – "Bounty Hacker" Room

Prepared by: Manoj Bora

Date: 15 September 2026

Classification: For Academic / Training Use Only

Document Control

Report Title	Penetration Test Report – TryHackMe Bounty Hacker
Target	10.48.144.93 (TryHackMe lab instance)
Tester	Manoj Bora
Engagement Type	Black-box, authorized training lab (TryHackMe)
Date of Testing	15 September 2026
Date of Report	15 September 2026
Classification	For Academic / Training Use Only

Executive Summary

This report documents the TryHackMe "Bounty Hacker" room, completed as part of my practical coursework for the Advanced Diploma in Cyber Security. The engagement was conducted as a black-box exercise – no prior credentials or system knowledge were given; all findings were obtained through active reconnaissance and exploitation.

I identified and chained three connected weaknesses on the target. First, the FTP service permitted anonymous login and exposed two files in the root directory: task.txt, which revealed the username lin, and locks.txt, a list of potential passwords. Second, the SSH service had no brute-force protection, allowing Hydra to identify the correct password for lin from that wordlist in under ten seconds. Third, the sudo policy granted lin unrestricted use of /bin/tar as root; using a well-known GTFOBins technique, this allowed a full root shell without any further exploitation.

Exploiting this chain took me from an unauthenticated external attacker to root on the box and yielded both flags – user.txt (THM{CR1M3_SyNd1C4T3}) and root.txt (THM{80UN7Y_h4cK3r}). Full technical details, command-by-command walkthroughs, and risk ratings for each issue are in the sections below.

Flags Captured

Flag	Value
User Flag (user.txt)	**THM{CR1M3_SyNd1C4T3}**
Root Flag (root.txt)	**THM{80UN7Y_h4cK3r}**

Scope & Objectives

Target: TryHackMe machine at 10.48.144.93.

Type of test: black-box – no credentials or information were given beforehand; everything was found during testing.

Objective: identify and exploit vulnerabilities to gain initial foothold on the target system, then escalate privileges to root, as required for the practical module of my Advanced Diploma in Cyber Security.

Methodology

The assessment followed the standard CEH five-phase approach, scoped to what was needed for this target:

- Reconnaissance – identifying the target IP and services it exposes.
- Scanning & Enumeration – port scanning, FTP anonymous login, and file retrieval.
- Gaining Access – SSH brute-force using discovered credentials.
- Privilege Escalation – identifying and abusing a misconfigured sudo rule to escalate from lin to root.
- Maintaining Access / Covering Tracks – not applicable; objective was flag capture only.

1. Reconnaissance & Port Scanning

After starting the room and receiving the target IP (10.48.144.93), I ran a TCP SYN scan using Nmap to identify which ports and services were exposed:

```
nmap -sS 10.48.144.93
```

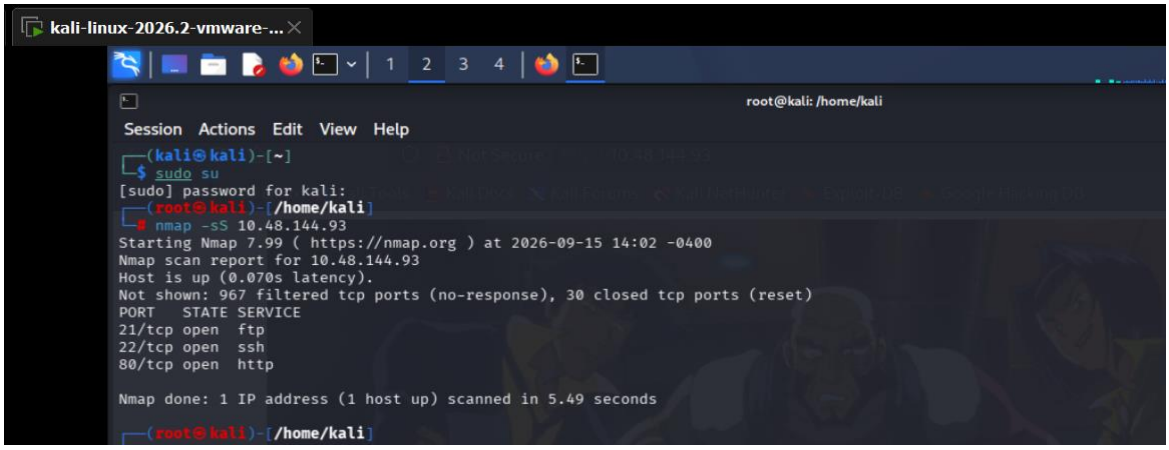


Figure 1 – Nmap scan output showing three open ports: 21/FTP, 22/SSH, and 80/HTTP

The scan returned three open ports: 21/tcp (FTP), 22/tcp (SSH), and 80/tcp (HTTP). Port 21 caught my attention immediately because FTP is often configured with anonymous access in CTF environments, making it the logical first target for enumeration.

2. FTP Enumeration – Anonymous Login & File Retrieval

I connected to the FTP service using the built-in ftp client and logged in as anonymous (no password required):

```
ftp 10.48.144.93
Name: anonymous
```

The server accepted the anonymous login and connected successfully. Running ls revealed two files in the FTP root directory: locks.txt and task.txt.

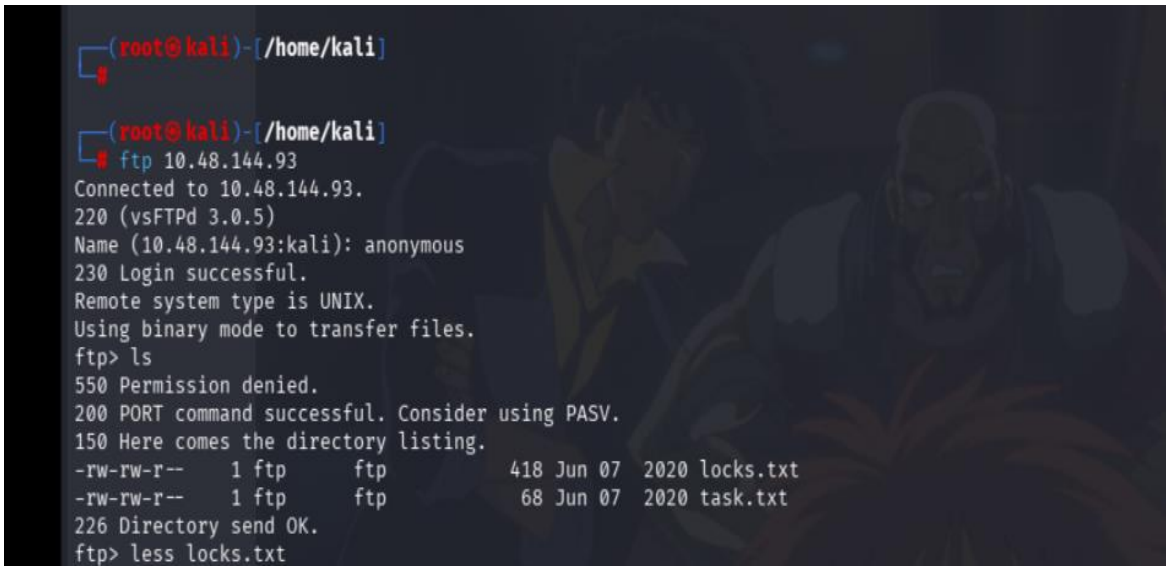


Figure 2 – Anonymous FTP login successful; directory listing shows locks.txt and task.txt

I downloaded both files using the get command:

```
get locks.txt
get task.txt
```

Examining task.txt revealed a short message that was signed at the end with the name lin – this became my assumed SSH username. The file locks.txt contained a list of 26 words, which I recognized as a password wordlist. With a username (lin) and a wordlist (locks.txt), SSH brute-forcing became the natural next step.

3. Credential Discovery – SSH Brute-Force with Hydra

I used Hydra to brute-force the SSH service using lin as the fixed username and locks.txt as the password wordlist:

```
hydra -l lin -P locks.txt ssh://10.48.144.93
```

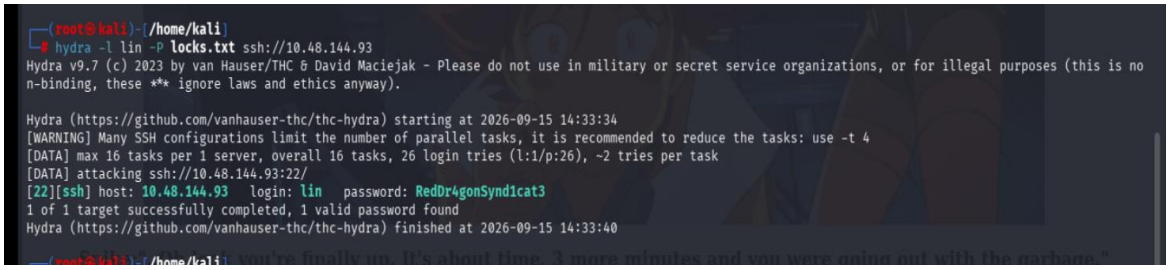


Figure 3 – Hydra brute-force against SSH; valid credentials found: lin / RedDr4gonSynd1cat3

Hydra completed in approximately six seconds and returned a single valid result:

```
[22][ssh] host: 10.48.144.93   login: lin   password: RedDr4gonSynd1cat3
```

The password RedDr4gonSynd1cat3 was present in locks.txt, confirming that the FTP-exposed wordlist contained the live SSH credential. This completed the credential discovery phase.

4. Initial Access – SSH Login as lin

Using the credentials found by Hydra, I logged into the target over SSH:

```
ssh lin@10.48.144.93
Password: RedDr4gonSynd1cat3
```

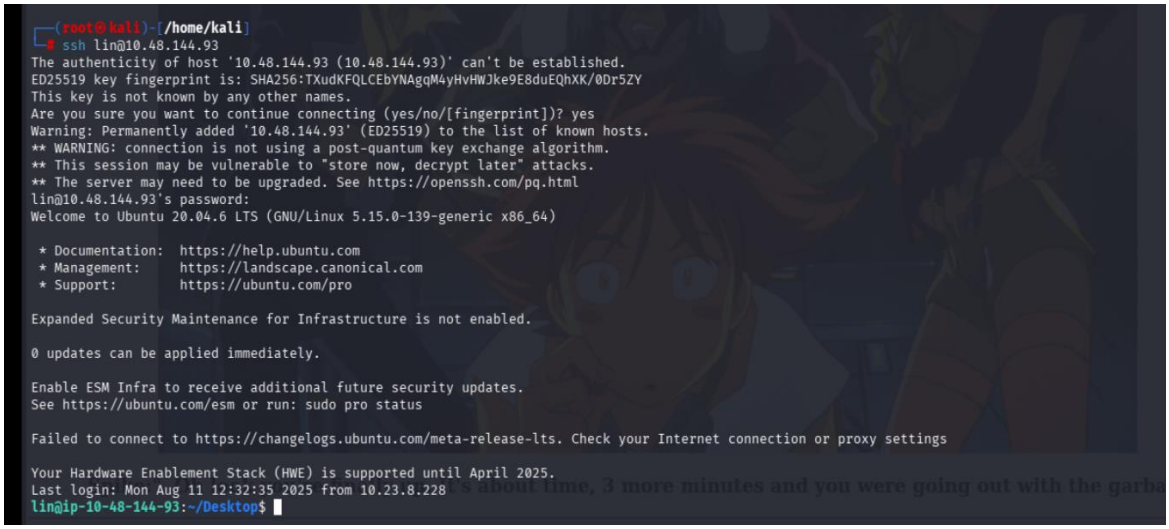


Figure 4 – Successful SSH login as lin; initial shell obtained on the target system (Ubuntu 20.04.6 LTS)

The SSH server accepted the credentials and presented an Ubuntu 20.04.6 LTS banner. I now had a live interactive shell on the target as user lin:

```
lin@ip-10-48-144-93:~/Desktop$
```

Shell prompt breakdown: lin is the logged-in user, ip-10-48-144-93 is the hostname, ~/Desktop resolves to /home/lin/Desktop (the tilde ~ always expands to the current user's home directory), and the $ symbol confirms this is a standard unprivileged user shell (# would indicate root).

5. User Flag Capture & Privilege Escalation

5.1 User Flag

After gaining initial access, I listed the contents of the Desktop and found user.txt:

```
ls
cat user.txt
```

This returned the first flag: THM{CR1M3_SyNd1C4T3}

Note: In TryHackMe CTF rooms on Linux, the user flag is typically found in the current user's home directory (/home/<username>). If the file is not immediately visible, the find command below can locate it:

```
find / -type f -name user.txt 2>/dev/null
```

5.2 Privilege Escalation – sudo Misconfiguration (tar)

With the user flag captured, I moved to the post-exploitation phase: escalating from lin to root. The standard first check for Linux privilege escalation is the sudo permissions list:

```
sudo -l
```

sudo -l lists every command the current user is permitted to run with sudo. The output showed:

```
User lin may run the following commands on ip-10-48-144-93:     (root) /bin/tar
```

This means lin can run /bin/tar as root without any restrictions. The tar utility is a file archiving tool, but GTFOBins documents a technique to abuse it for privilege escalation. I used the following command from GTFOBins, prefixing the tar path with sudo:

```
sudo /bin/tar cf /dev/null /dev/null --checkpoint=1 --checkpoint-
action=exec=/bin/sh
```

Command breakdown: cf /dev/null /dev/null creates an empty archive written to /dev/null (discarded). --checkpoint=1 fires a checkpoint after every record. --checkpoint-action=exec=/bin/sh executes /bin/sh at each checkpoint. Since the entire command runs as root (via sudo), the spawned shell inherits root privileges.

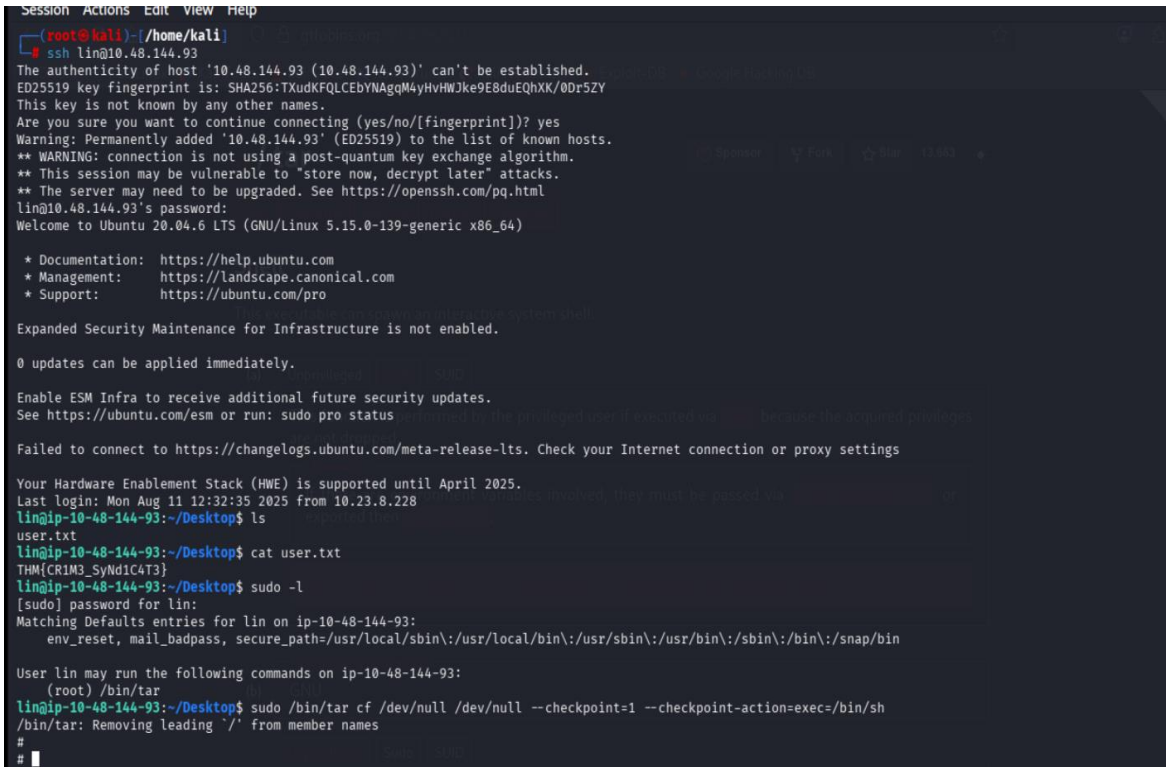


Figure 5 – cat user.txt returning the user flag; sudo -l revealing the tar permission; tar exploit spawning a root shell (#)

The shell prompt changed from $ to #, confirming successful escalation to root.

6. Root Flag Capture

With root shell access confirmed, I navigated to the root user's home directory and retrieved the final flag:

```
cd /root
ls
cat root.txt
```

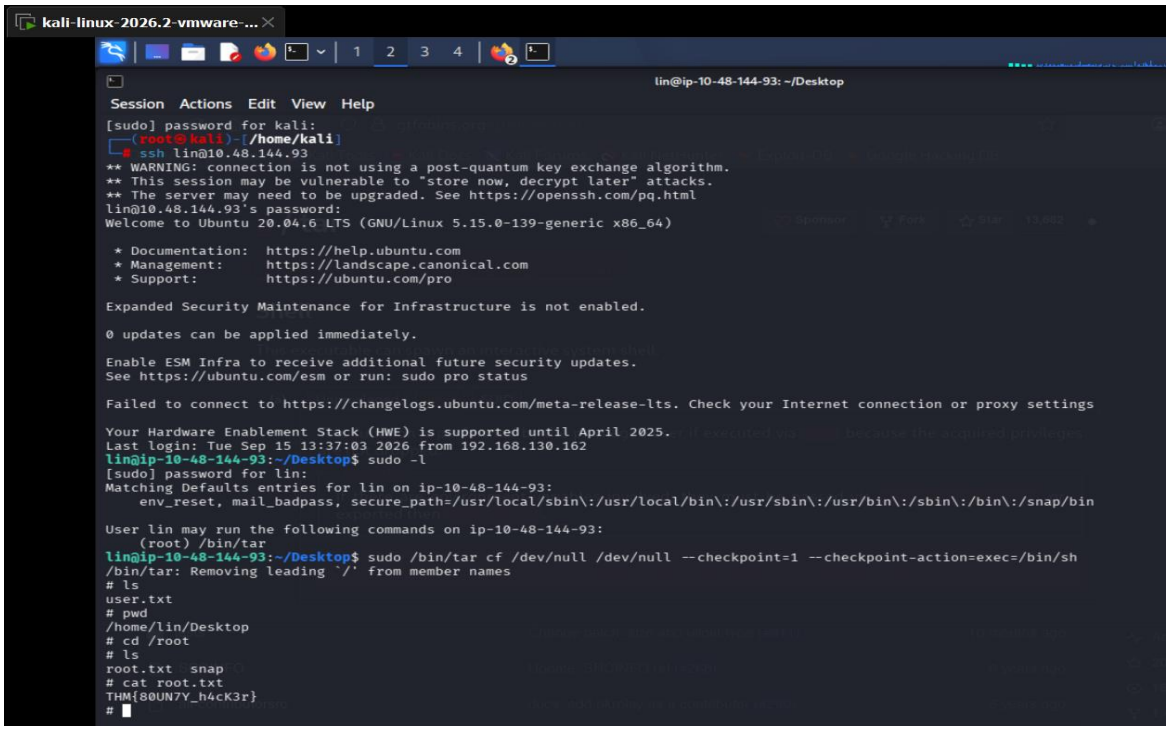


Figure 6 – Root shell confirmed; navigating to /root and reading root.txt for the final flag: THM{80UN7Y_h4cK3r}

The final flag was returned: THM{80UN7Y_h4cK3r}

Note: In TryHackMe CTF rooms on Linux, the root flag is always stored in /root/root.txt. In OSCP terminology, this file is equivalent to proof.txt on Linux and can be found at C:\Users\Administrator\Desktop\proof.txt on Windows targets.

Tools Used

Tool	Type	Purpose
Nmap	Network scanner	TCP SYN scan to identify open ports and exposed services on the target host
FTP Client	File transfer client	Anonymous FTP login to enumerate and download files from the target
Hydra	Credential brute-forcer	SSH brute-force attack using locks.txt as a password wordlist
SSH (OpenSSH)	Remote access tool	Gaining initial shell access as user lin with discovered credentials
sudo / GTFOBins	Privilege escalation	Identifying sudo misconfiguration and exploiting tar via GTFOBins to escalate to root

Findings

Three connected weaknesses were identified during this room. Each one fed directly into the next, forming a complete attack chain from unauthenticated external access to root. Severity ratings and CVSS scores are in the Risk Summary section.

Finding 1: Anonymous FTP Access Exposing Sensitive Files

The FTP service on port 21 accepted anonymous logins with no password. This allowed any unauthenticated attacker to browse the FTP server's directory and download its contents. The two files present – task.txt and locks.txt – directly provided the username and password wordlist needed for the next stage of the attack.

What I did:

- Connected with: ftp 10.48.144.93 and logged in as anonymous.
- Ran ls – found locks.txt and task.txt.
- Downloaded both files using get.
- task.txt revealed the username lin (signed at the end of the message).
- locks.txt contained 26 potential passwords, one of which was the live SSH credential.

Why it matters: Sensitive operational files (usernames, password wordlists) should never be stored on a publicly accessible FTP server. Anonymous FTP access removes all authentication barriers, meaning any internet-connected machine can retrieve these files without any credentials.

How to fix it: Disable anonymous FTP access entirely. If FTP must be used, enforce authentication and restrict access by IP. For secure file transfers, replace FTP with SFTP or SCP. Never store credential-related files in FTP directories.

Finding 2: Weak SSH Password Leading to Brute-Force Compromise

The SSH service on port 22 had no brute-force protection (no fail2ban, no rate-limiting, no account lockout). Combined with a weak password that appeared in the FTP-exposed wordlist, Hydra was able to recover the valid credential for lin in under ten seconds.

What I did:

- Ran: hydra -l lin -P locks.txt ssh://10.48.144.93
- Hydra tested 26 passwords and returned the valid one: RedDr4gonSynd1cat3.
- Logged into SSH: ssh lin@10.48.144.93 with the discovered password.

- Confirmed an interactive shell as lin on the target.

Why it matters: A password is only as strong as it is unpredictable. RedDr4gonSynd1cat3 is a substitution-based password (letters replaced with look-alike numbers and symbols) that appeared in an attacker-accessible wordlist on the same machine – making it effectively no protection at all. Without rate-limiting on SSH, an attacker can test every entry in a wordlist with zero friction.

How to fix it: Use long, randomly generated passwords or SSH key-based authentication instead. Enable fail2ban or equivalent to rate-limit and temporarily ban IPs after repeated failed logins. Never use the same file that contains a password as an accessible resource on any networked service.

Finding 3: Privilege Escalation via sudo tar Misconfiguration

The sudoers policy granted user lin unlimited sudo rights over /bin/tar with no restrictions. The tar utility has a well-known GTFOBins technique (--checkpoint-action=exec) that can execute arbitrary commands. Since it was run via sudo, the spawned shell inherited root privileges.

What I did:

- Ran sudo -l – revealed: (root) /bin/tar with no password restriction.
- Consulted GTFOBins for the tar sudo technique.
- Ran: sudo /bin/tar cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
- Shell prompt changed from $ to #, confirming root access.
- Navigated to /root and read root.txt: THM{80UN7Y_h4cK3r}.

Why it matters: Granting sudo access to a general-purpose utility like tar is equivalent to granting unrestricted root – the utility's own flags allow it to spawn arbitrary shells or execute any command. An attacker with any foothold on the box (even as a low-privilege user) can immediately become root with a single command.

How to fix it: Remove the tar entry from the sudoers file. Audit all sudo rules with sudo -l across every user account and remove any that grant access to general-purpose utilities (tar, python, find, vim, etc.) listed on GTFOBins. If elevated access to archiving is genuinely needed, use a narrowly scoped wrapper script rather than the binary directly.

Risk Summary

Finding	CVSS 3.1 Base	Assessed Severity	Category
Anonymous FTP Access → Credential Leakage	**7.5 (High)**	**High**	CWE-287 Improper Authentication
SSH Brute-Force via Weak Password	**9.8 (Critical)**	**Critical**	CWE-521 Weak Password Requirements
Privilege Escalation via sudo tar Misconfiguration	**7.8 (High)**	**High**	CWE-269 Improper Privilege Management

Conclusion & Recommendations

The Bounty Hacker room demonstrated a clean, end-to-end attack chain that required no advanced techniques – just methodical enumeration and standard tooling. Every step flowed directly from the previous one: anonymous FTP gave the username and wordlist, the wordlist gave the SSH password, and the SSH foothold gave the sudo rule that opened the door to root.

The three core recommendations are:

- Disable anonymous FTP – or remove FTP entirely in favour of SFTP/SCP. Never leave credential-related files accessible on a public-facing service.
- Harden SSH – enforce key-based authentication, disable password logins where possible, and deploy fail2ban or equivalent to block brute-force attempts at the network level.
- Audit and minimise sudo rules – follow the principle of least privilege. Remove any sudo permission that grants access to binaries listed on GTFOBins. Run periodic audits with sudo -l across all non-root accounts.

These three changes together would have broken the attack chain at every stage. None of them require significant effort – they are standard hardening steps that are consistently recommended in CIS Benchmarks and industry best practice guides.